

Redux Interview Questions & Answers

Redux Toolkit • Async Redux • RTK Query • Performance • Senior React Interview Preparation

Redux Interview Questions & Answers

■ 1. What is Redux?

Answer:

Redux is a predictable state management library used to manage global application state.

The basic Redux flow is:

```
Component
↓
dispatch(action)
↓
Reducer
↓
Store updates
↓
Component gets updated state
```

Redux is useful when multiple components need access to the same state.

■ 2. What are the core concepts of Redux?

The main concepts are:

Store – holds application state.

Action – describes what happened.

Reducer – determines how state changes.

Dispatch – sends an action to Redux.

Selector – reads data from the store.

Middleware – handles logic between dispatch and reducer.

Example:

```
dispatch({
  type: "counter/increment"
});
```

The reducer receives this action and updates the state.

■ 3. What is a Redux Store?

The store is the central place where Redux keeps application state.

With Redux Toolkit:

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterSlice";
export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

The store contains:

```
{
  counter: {
    value: 0
  }
}
```

■ 4. What is an Action?

An action is a JavaScript object describing what happened.

Example:

```
{
  type: "counter/increment"
}
```

An action can also contain data using payload:

```
{
  type: "counter/incrementByAmount",
  payload: 5
}
```

■ 5. What is a Reducer?

A reducer is a function that determines how the state changes based on an action.

Traditional Redux:

```
const initialState = {
  count: 0,
};
function counterReducer(state = initialState, action) {
  switch (action.type) {
```

case "increment":

```
    return {
      ...state,
      count: state.count + 1,
    };
    default:
      return state;
  }
}
```

The reducer should be predictable and should not perform side effects.

■ 6. What is Redux Toolkit?

Redux Toolkit (RTK) is the official recommended way to write Redux logic.

It reduces Redux boilerplate by providing utilities such as:

configureStore

createSlice

createAsyncThunk

createEntityAdapter

RTK Query

Instead of writing:

```
const INCREMENT = "counter/increment";
const increment = () => ({
  type: INCREMENT,
});
```

RTK allows:

```
const counterSlice = createSlice({
  name: "counter",
  initialState: {
    count: 0,
  },
  reducers: {
    increment(state) {
      state.count++;
    },
  },
});
```

```
},  
});
```

■ 7. What is createSlice?

createSlice() is one of the most important Redux Toolkit APIs.

It combines:

initial state

reducers

action creators

action types

Example:

```
import { createSlice } from "@reduxjs/toolkit";  
const counterSlice = createSlice({  
  name: "counter",  
  initialState: {  
    count: 0,  
  },  
  reducers: {  
    increment(state) {  
      state.count += 1;  
    },  
    decrement(state) {  
      state.count -= 1;  
    },  
    incrementByAmount(state, action) {  
      state.count += action.payload;  
    },  
  },  
});  
export const {  
  

```

increment,

decrement,

incrementByAmount,

```
  } = counterSlice.actions;  
  export default counterSlice.reducer;
```

This generates action creators automatically.

■ 8. Why can we mutate state inside Redux Toolkit reducers?

This is a very common interview question.

Normally Redux state should be updated immutably.

However, Redux Toolkit uses Immer internally.

```
Therefore:  
increment(state) {  
  state.count++;  
}
```

is converted internally into an immutable state update.

```
Conceptually:  
return {
```

```
  ...state,
```

```
  count: state.count + 1,  
};
```

So you're writing mutation-style code, but Redux Toolkit produces an immutable update.

■ 9. How do you configure Redux in React?

Step 1 — Create slice

```
// counterSlice.js
import { createSlice } from "@reduxjs/toolkit";
const counterSlice = createSlice({
  name: "counter",
  initialState: {
    count: 0,
  },
  reducers: {
    increment(state) {
      state.count++;
    },
    decrement(state) {
      state.count--;
    },
  },
});
export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Step 2 — Create store

```
// store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterSlice";
export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

Step 3 — Provide store to React

```
import { Provider } from "react-redux";
import { store } from "./store";
function App() {
  return (
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}
export default App;
```

Step 4 — Access state

```
import { useDispatch, useSelector } from "react-redux";
import { increment, decrement } from "./counterSlice";
function Counter() {
  const count = useSelector(
    (state) => state.counter.count
  );
  const dispatch = useDispatch();
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => dispatch(increment())}>
+
      </button>
      <button onClick={() => dispatch(decrement())}>
-
      </button>
    </div>
  );
}
```

■ 10. What is dispatch?

dispatch() sends an action to the Redux store.

Example:

```
dispatch(increment());
```

The flow is:

```
dispatch()
```

↓
Middleware
↓
Reducer
↓
State update
↓
React re-render

With payload:

```
dispatch(incrementByAmount(10));
```

■ 11. What is useSelector?

```
useSelector() reads data from Redux state.  
const count = useSelector(  
  (state) => state.counter.count  
);
```

If the selected value changes, the component can re-render.

■ 12. What is useDispatch?

```
useDispatch() gives you access to Redux's dispatch function.  
const dispatch = useDispatch();  
dispatch(increment());
```

■ 13. What is payload?

Payload is the data sent along with an action.

Example:

```
dispatch(addUser({  
  id: 1,  
  name: "Priyanka"  
}));
```

The reducer receives:

```
addUser(state, action) {  
  console.log(action.payload);  
}  
Output:  
{  
  id: 1,  
  name: "Priyanka"  
}
```

■ 14. How do you pass data to a Redux action?

Using payload.

```
const userSlice = createSlice({  
  name: "user",  
  initialState: {  
    name: "",  
  },  
  reducers: {  
    setName(state, action) {  
      state.name = action.payload;  
    },  
  },  
});  
export const { setName } = userSlice.actions;
```

Component:

```
dispatch(setName("Priyanka"));
```

Reducer receives:

```
action.payload  
// "Priyanka"
```

■ 15. What is Redux middleware?

Middleware sits between:

```
dispatch(action)
↓
```

middleware

```
↓
```

reducer

Middleware is useful for:

asynchronous operations

logging

analytics

API calls

error handling

Common Redux middleware includes:

Redux Thunk

custom middleware

logging middleware

■ 16. What is Redux Thunk?

Redux Thunk allows you to dispatch functions instead of only plain objects.

Example:

```
const fetchUsers = () => async (dispatch) => {
  dispatch(usersLoading());
  const response = await fetch("/api/users");
  const users = await response.json();
  dispatch(usersLoaded(users));
};
Then:
dispatch(fetchUsers());
```

Redux Toolkit includes thunk middleware by default.

■ 17. What is createAsyncThunk?

`createAsyncThunk()` is Redux Toolkit's convenient way to handle asynchronous operations.

Example:

```
import {
  createAsyncThunk,
  createSlice,
} from "@reduxjs/toolkit";
export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async () => {
    const response = await fetch("/api/users");
    return response.json();
  }
);
```

Then handle the states:

```
const userSlice = createSlice({
  name: "users",
  initialState: {
    users: [],
    loading: false,
    error: null,
```

```

},
reducers: {},
extraReducers: (builder) => {

```

builder

```

.addCase(fetchUsers.pending, (state) => {
  state.loading = true;
})
.addCase(fetchUsers.fulfilled, (state, action) => {
  state.loading = false;
  state.users = action.payload;
})
.addCase(fetchUsers.rejected, (state, action) => {
  state.loading = false;
  state.error = action.error.message;
});
},
});

```

Component:

```

const dispatch = useDispatch();
useEffect(() => {
  dispatch(fetchUsers());
}, [dispatch]);

```

■ 18. Explain pending, fulfilled and rejected

When using createAsyncThunk:

```

dispatch(fetchUsers())
↓

```

pending

```

↓
API call
■ ■

```

fulfilled rejected

Pending

```

.addCase(fetchUsers.pending, (state) => {
  state.loading = true;
})

```

Fulfilled

```

.addCase(fetchUsers.fulfilled, (state, action) => {
  state.loading = false;
  state.users = action.payload;
})

```

Rejected

```

.addCase(fetchUsers.rejected, (state, action) => {
  state.loading = false;
  state.error = action.error.message;
})

```

■ 19. What is extraReducers?

extraReducers allows a slice to respond to actions that were not generated by that slice's reducers field.

A common example is createAsyncThunk.

```

extraReducers: (builder) => {

```

builder

```

.addCase(fetchUsers.pending, ...)
.addCase(fetchUsers.fulfilled, ...)
.addCase(fetchUsers.rejected, ...);
}

```

■ 20. Reducers vs extraReducers?

■ 21. What is a selector?

A selector is a function used to retrieve specific information from Redux state.

Instead of:

```
const users = useSelector(
  (state) => state.users.users
);
```

You can create:

```
const selectUsers = (state) => state.users.users;
Then:
const users = useSelector(selectUsers);
```

Selectors help keep state-access logic reusable.

■ 22. What is a memoized selector?

A memoized selector avoids recalculating derived data when its inputs haven't changed.

A common library is Reselect, which is integrated into the Redux ecosystem.

Example:

```
import { createSelector } from "@reduxjs/toolkit";
const selectUsers = (state) => state.users.users;
const selectActiveUsers = createSelector(
  [selectUsers],
  (users) =>
    users.filter((user) => user.active)
);
Then:
const activeUsers = useSelector(selectActiveUsers);
```

This is useful when derived calculations are expensive.

■ 23. What is Redux DevTools?

Redux DevTools helps developers inspect:

actions

state changes

previous state

next state

action payloads

time-travel debugging

With Redux Toolkit:

```
configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

DevTools integration is enabled by default in development.

■ 24. What is normalization in Redux?

Normalization means storing relational data in a structured form instead of deeply nested objects.

```
Bad:
{
  posts: [
    {
      id: 1,
      author: {
        id: 10,
```



```

    name: "John"
  }
}
]
}
Normalized:
{
  users: {
    10: {
      id: 10,
      name: "John"
    }
  },
  posts: {
    1: {
      id: 1,
      authorId: 10
    }
  }
}
}

```

This reduces duplication and makes updates easier.

Redux Toolkit provides:

`createEntityAdapter()`

for normalized collections.

■ 25. What is RTK Query?

RTK Query is Redux Toolkit's data-fetching and caching solution.

It can handle:

API requests

caching

loading states

errors

refetching

cache invalidation

Example:

```

import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
export const api = createApi({
  reducerPath: "api",
  baseQuery: fetchBaseQuery({
    baseUrl: "/api",
  }),
  endpoints: (builder) => ({
    getUsers: builder.query({
      query: () => "/users",
    }),
  }),
});
export const {
  useGetUsersQuery,
} = api;

```

Component:

```

function Users() {
  const {
    data,
    isLoading,

```

```

error,

} = useGetUsersQuery();
if (isLoading) {
  return <p>Loading...</p>;
}
if (error) {
  return <p>Error</p>;
}
return (
  <ul>
    {data.map((user) => (
      <li key={user.id}>
        {user.name}
      </li>
    ))}
  </ul>
);
}

```

■ 26. Redux vs Context API?

Context is not necessarily a replacement for Redux.

■ 27. Redux vs Zustand?

Since you know Zustand, this is especially important for your interviews.

Example Zustand:

```

const useCounterStore = create((set) => ({
  count: 0,
  increment: () =>
    set((state) => ({
      count: state.count + 1,
    })),
}));

```

Redux Toolkit:

```

const counterSlice = createSlice({
  name: "counter",
  initialState: {
    count: 0,
  },
  reducers: {
    increment(state) {
      state.count++;
    },
  },
});

```

Interview answer

"I would choose Redux Toolkit when the application has complex global state, multiple developers, standardized patterns, extensive middleware requirements, or when RTK Query and Redux DevTools are valuable. For smaller applications or simpler shared state, Zustand can provide a much lighter solution."

■ 28. How do you handle loading and error states in Redux?

A common pattern is:

```

initialState: {
  data: [],
  loading: false,
  error: null,
}
Then:
pending:
state.loading = true;
state.error = null;
Success:
state.loading = false;
state.data = action.payload;
Error:
state.loading = false;

```

```
state.error = action.error.message;
```

This creates a predictable UI state:

```
loading
↓
success / error
```

■ 29. Can reducers make API calls?

No.

Reducers must be pure functions.

Don't do:

```
function reducer(state, action) {
  fetch("/api/users"); // ■
}
```

Instead use:

createAsyncThunk

RTK Query

middleware

external async logic

■ 30. Why should reducers be pure?

A pure reducer:

same state + same action

↓

same result

Example:

```
function reducer(state, action) {
  if (action.type === "increment") {
    return {
      ...state,
      count: state.count + 1,
    };
  }
  return state;
}
```

Reducers should not perform:

API calls

timers

random operations

direct DOM manipulation

other side effects

■ 31. What is immutability in Redux?

Instead of modifying an existing state object:

```
state.user.name = "John";
```

traditional Redux expects a new state reference:

```
return {
```

```
...state,
  user: {
...state.user,
  name: "John",
},
};
```

Redux Toolkit's Immer allows:

```
state.user.name = "John";
```

inside RTK reducers while maintaining immutable state semantics.

■ 32. Why does Redux use immutable state?

Immutability helps Redux:

detect changes efficiently

enable predictable updates

support time-travel debugging

prevent accidental state mutations

optimize React rendering

■ 33. What happens when dispatch is called?

```
For:
dispatch(increment());
```

the flow is approximately:

```
Component
↓
dispatch()
↓
Middleware
↓
Reducer
↓
New Redux state
↓
Subscribers notified
↓
useSelector checks selected value
↓
Component re-renders if needed
```

This is an excellent interview explanation.

■ 34. How can you prevent unnecessary Redux re-renders?

Use:

1. Specific selectors

Instead of:

```
const state = useSelector((state) => state);
prefer:
const count = useSelector(
  (state) => state.counter.count
);
```

2. Memoized selectors

```
createSelector(...)
```

3. Split components

Instead of one huge component consuming the entire Redux state, allow smaller components to subscribe to the specific pieces they need.

4. Avoid creating unnecessary objects in selectors

For example:

```
useSelector((state) => ({
  name: state.user.name,
  email: state.user.email,
}));
```

can cause reference changes unless handled appropriately.

■ 35. How would you structure Redux in a large application?

A common feature-based structure:

src/

■

■ ■ ■ ■ app/

■ ■ ■ ■ store.js

■

■ ■ ■ ■ features/

■ ■ ■ ■ auth/

■ ■ ■ ■ authSlice.js

■ ■ ■ ■ authSelectors.js

■ ■ ■ ■ authApi.js

■ ■

■ ■ ■ ■ users/

■ ■ ■ ■ usersSlice.js

■ ■ ■ ■ usersSelectors.js

■ ■ ■ ■ usersApi.js

■ ■

■ ■ ■ ■ products/

■ ■ ■ ■ productsSlice.js

■ ■ ■ ■ productsSelectors.js

■ ■ ■ ■ productsApi.js

■

■ ■ ■ ■ components/

Feature-based organization scales better than putting every reducer and action in separate global folders.

■ 36. How would you implement authentication using Redux?

Example state:

```
const initialState = {
  user: null,
  token: null,
  isAuthenticated: false,
};
Slice:
const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers: {
    login(state, action) {
      state.user = action.payload.user;
```

```

state.token = action.payload.token;
state.isAuthenticated = true;
},
logout(state) {
state.user = null;
state.token = null;
state.isAuthenticated = false;
},
},
});
Login:
dispatch(
login({

```

user,

token,

```

})
);
Logout:
dispatch(logout());

```

For production authentication, token storage and security should be designed carefully rather than blindly storing sensitive tokens in Redux/localStorage.

■ 37. Redux Toolkit interview coding question

Question:

Create a counter that supports:

increment

decrement

reset

increment by amount

Answer:

```

import { createSlice } from "@reduxjs/toolkit";
const counterSlice = createSlice({
name: "counter",
initialState: {
value: 0,
},
reducers: {
increment(state) {
state.value += 1;
},
decrement(state) {
state.value -= 1;
},
reset(state) {
state.value = 0;
},
incrementByAmount(state, action) {
state.value += action.payload;
},
},
});
export const {

```

increment,

decrement,

reset,

incrementByAmount,

```

} = counterSlice.actions;
export default counterSlice.reducer;

```

Component:

```

function Counter() {
  const count = useSelector(
    (state) => state.counter.value
  );
  const dispatch = useDispatch();
  return (
    <>
    <h2>{count}</h2>
    <button onClick={() => dispatch(increment())}>
+
    </button>
    <button onClick={() => dispatch(decrement())}>
-
    </button>
    <button onClick={() => dispatch(reset())}>
Reset
    </button>
    <button
    onClick={() =>
    dispatch(incrementByAmount(10))
    }
>
+10
    </button>
  </>
);
}

```

■ 38. Redux API interview coding question

Question:

Fetch users from an API and handle loading, success and error states.

```

export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async (_, thunkAPI) => {
    try {
      const response = await fetch("/api/users");
      if (!response.ok) {
        throw new Error("Failed to fetch users");
      }
      return await response.json();
    } catch (error) {
      return thunkAPI.rejectWithValue(
error.message
      );
    }
  });
Slice:
const usersSlice = createSlice({
  name: "users",
  initialState: {
    data: [],
    loading: false,
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
builder
    .addCase(fetchUsers.pending, (state) => {
      state.loading = true;
      state.error = null;
    })

```

```

.addCase(fetchUsers.fulfilled, (state, action) => {
  state.loading = false;
  state.data = action.payload;
})
.addCase(fetchUsers.rejected, (state, action) => {
  state.loading = false;
  state.error = action.payload;
});
},
});

```

This is a very good coding example to practice for a Senior React interview.

One answer worth memorizing

If an interviewer asks "Explain Redux in your project", a strong answer is:

"I use Redux Toolkit for managing complex global application state. I organize the state by feature using slices. Each slice contains its initial state and reducers, while asynchronous operations can be handled using `createAsyncThunk` or `RTK Query` depending on the use case. Components access state using `useSelector` and update it using `useDispatch`. I use selectors to keep state access reusable and optimize re-renders. For API-heavy use cases, `RTK Query` can handle caching, loading states, refetching, and cache invalidation. For simpler local or shared state, I would consider `Zustand` instead of introducing Redux unnecessarily."